

UNIVERSIDAD RAFAEL LANDÍVAR
INGENIERÍA EN INFORMÁTICA Y SISTEMAS
CUARTO SEMESTRE
ING. MANUEL ROJAS
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS
PROYECTO I
PABLO DAVID QUIJIVIX SUCHI CARNÉ: 1578125



Universidad
Rafael Landívar
Tradición Jesuita en Guatemala

DOCUMENTACIÓN TÉCNICA Y MANUAL DE USUARIO DEL
SISTEMA DE APLICCIÓN DE ESCRITORIO CON GESTIÓN DE
CONFIGURACIÓN DE USUARIO

Quetzaltenango 16 de septiembre 2026.

1. Descripción General del Proyecto

El sistema consiste en una aplicación de escritorio desarrollada en Python bajo un enfoque de programación modular. Su objetivo principal es garantizar la persistencia robusta, segura y consistente de un archivo de configuración de usuario en formato JSON ya que fue este el tipo de archivo que se eligió para este desarrollo. El diseño arquitectónico desacopla la interfaz gráfica de la lógica de almacenamiento, incorporando mecanismos avanzados de escritura atómica mediante archivos temporales (.tmp), respaldos automáticos (.bak), codificación estricta de caracteres en UTF-8 y un control exhaustivo de excepciones ante fallos del sistema de archivos.

2. Justificación Técnica del Formato de Almacenamiento

Para el almacenamiento de los ajustes de usuario se seleccionó estrictamente el formato JSON, evaluándolo frente a alternativas comunes como archivos de texto plano personalizados o CSV:

- Formato Elegido (JSON):

- Tamaño: Compacto y eficiente para estructuras de datos clave-valor de configuración.

- Legibilidad: Altamente legible tanto para quienes trabajen directamente con el código como para analizadores automáticos.

- Facilidad de parseo: Soporte nativo e inmediato mediante la librería integrada json de Python, lo que reduce la complejidad del código.

- Robustez ante corrupción: Permite validar esquemas y detectar errores de sintaxis de forma inequívoca mediante excepciones controladas.

- Alternativa Descartada (Texto Plano Estructurado con Delimitadores / CSV):

Se descartó debido a la complejidad en el análisis de tipos de datos anidados o configuraciones flexibles, así como a la alta vulnerabilidad a la corrupción ante modificaciones manuales en la estructura de separación por comas o líneas.

3. Implementación de Escritura Segura y Respaldo

El proceso de persistencia implementa una estrategia de escritura atómica y de prevención de pérdidas de datos basada en dos etapas críticas:

- Copia de Respaldo Previa (.bak): Antes de sobrescribir cualquier archivo existente, se genera una copia de seguridad exacta con la extensión .bak utilizando la utilidad de copiado del sistema.

-Escritura Atómica mediante Archivo Temporal (.tmp): Los datos no se escriben directamente sobre el archivo de producción para evitar corrupciones en caso de un cierre abrupto del software; los datos se escriben primero en un archivo temporal config.json.tmp y, una vez finalizada con éxito la escritura, se reemplaza de manera atómica el archivo original.

- Fragmento de código relevante integrado en el sistema:

```
if os.path.exists(self.ruta_archivo):  
    shutil.copy2(self.ruta_archivo, self.ruta_respaldo)  
    with open(self.ruta_temporal, "w", encoding="utf-8") as archivo_tmp:  
        json.dump(nuevos_datos, archivo_tmp, indent=4, ensure_ascii=False)  
    os.replace(self.ruta_temporal, self.ruta_archivo)
```

4. Manejo Robusto de Errores de Archivo

La aplicación incorpora bloques de control de excepciones explícitos para asegurar la continuidad operativa y evitar interrupciones por tracebacks no capturados:

- Archivo Ausente: Si el archivo de configuración no existe en el directorio al iniciar, el sistema intercepta el estado y carga de forma automática los valores predeterminados definidos por el sistema sin lanzar excepciones.

- Archivo Corrupto o Formato Inválido: Ante errores de decodificación JSON (JSONDecodeError), el sistema intercepta el fallo, emite un aviso en consola y procede a restaurar de forma automática la configuración válida a partir del archivo de respaldo (.bak).

- Falta de Permisos de Lectura o Escritura: Las restricciones del sistema operativo en cuanto a permisos (PermissionError) son capturadas explícitamente, degradando la ejecución a un comportamiento seguro predeterminado o notificando la imposibilidad de almacenamiento en disco sin colapsar la interfaz gráfica.

5. Ejemplo del Archivo de Configuración Generado (Soporte UTF-8)

El archivo generado garantiza de manera estricta el uso de codificación UTF-8, permitiendo el almacenamiento y recuperación correcta de nombres, caracteres especiales, tildes y ñes:

```
{  
  "nombre_usuario": "María José Quijivix",  
  "tema_interfaz": "oscuro",  
  "idioma": "es-ES",  
  "tamano_fuente": 14,  
  "color_barra_menu": "#2c3e50",  
  "color_letra": "#ecf0f1",  
  "foto_perfil": "C:/Users/Usuario/Pictures/perfil.png"  
}
```

6. Estructura Modular del Sistema

El código fuente se encuentra dividido en tres módulos independientes para mantener el orden, la escalabilidad y la separación de responsabilidades:

- main.py: Módulo de ejecución y desarrollo general.
- persistencia.py: Módulo encargado de la gestión de archivos, almacenamiento JSON, seguridad atómica y manejo de errores.
- interfaz.py: Módulo responsable de la presentación gráfica mediante Tkinter, menús y selectores visuales (Implementación de Interfaz gráfica sencilla).

7. Documentación de Funciones por Módulo

Módulo 1: main.py (Orquestador Principal)

- main()

Objetivo: Funcionar como el punto de entrada principal para la ejecución de la aplicación de escritorio.

Funcionamiento Específico: Instancia la clase GestorPersistencia pasando la ruta del archivo de configuración (config.json), inicializa la aplicación gráfica principal a través de AplicacionPrincipal y ejecuta el bucle de eventos (mainloop()) que mantiene activa la interfaz de usuario.

Módulo 2: persistencia.py (Gestión de Archivos y Persistencia)

- GestorPersistencia.__init__(ruta_archivo="config.json")

Objetivo: Inicializar las rutas de los archivos críticos del sistema y definir los valores de configuración predeterminados.

Funcionamiento Específico: Configura las rutas para el archivo principal (config.json), el archivo temporal de escritura segura (config.json.tmp) y el archivo de respaldo (config.json.bak). Asimismo, establece un diccionario con los parámetros por defecto de la aplicación (nombre de usuario, tema, idioma, tamaño de fuente, colores y foto de perfil).

- GestorPersistencia.cargar_configuracion()

Objetivo: Leer, decodificar y recuperar la configuración almacenada en el disco bajo codificación UTF-8.

Funcionamiento Específico: Verifica la existencia física del archivo principal. Si está ausente, retorna los valores por defecto. Si el archivo existe, abre el flujo en modo lectura con codificación UTF-8 y parsea el contenido JSON. En caso de detectar corrupción de datos o formato inválido (JSONDecodeError), invoca automáticamente el mecanismo de recuperación de respaldo (_restaurar_desde_respaldo). Si se presentan restricciones de permisos de lectura (PermissionError), degrada de forma segura al comportamiento predeterminado sin generar interrupciones abruptas (tracebacks).

- GestorPersistencia.guardar_configuracion(nuevos_datos)

Objetivo: Almacenar de forma atómica y segura los nuevos ajustes configurados por el usuario.

Funcionamiento Específico: Implementa una estrategia de escritura segura (Write-Ahead). Primero, crea una copia de respaldo exacta (.bak) del archivo original si este existe. Segundo, escribe los datos serializados en JSON en un archivo temporal (.tmp) utilizando codificación UTF-8 y un formato indentado. Tercero, reemplaza de manera atómica el archivo original con el archivo temporal (os.replace o os.rename). Si ocurre un error de permisos o de disco durante el proceso, intercepta la excepción, limpia el archivo temporal residual y retorna un indicador de fallo.

- GestorPersistencia._restaurar_desde_respaldo()

Objetivo: Proveer tolerancia a fallos restaurando un estado previo válido del sistema a partir de la copia de seguridad.

Funcionamiento Específico: Comprueba la existencia del archivo de respaldo (.bak). Si se encuentra disponible, lo lee e interpreta para devolver una configuración válida al usuario.

Si el respaldo tampoco es accesible o está dañado, retorna los valores por defecto del sistema.

- GestorPersistencia._limpiar_temporal()

Objetivo: Asegurar la limpieza del sistema de archivos eliminando residuos temporales tras un fallo de escritura.

Funcionamiento Específico: Verifica si el archivo temporal (.tmp) persiste en el disco y procede a eliminarlo de forma controlada para evitar la acumulación de archivos basura.

Módulo 3: interfaz.py (Capa de Presentación y Gráfica)

- VentanaConfiguracion.__init__(master, gestor_persistencia, callback_actualizar)

Objetivo: Construir la interfaz secundaria destinada a la edición de los ajustes de usuario (Settings).

Funcionamiento Específico: Inicializa una ventana de tipo Toplevel, carga la configuración actual mediante el gestor de persistencia y despliega los campos interactivos correspondientes (entradas de texto, selectores de opciones, selectores de color y explorador de archivos).

- VentanaConfiguracion._crear_widgets()

Objetivo: Diseñar y posicionar los componentes visuales interactivos dentro de la ventana de configuración.

Funcionamiento Específico: Instancia etiquetas y campos de entrada para el nombre de usuario y el idioma; botones de opción (radiobuttons) para el tema visual (claro/oscuro); un selector numérico (spinbox) para el tamaño de la fuente; botones interactivos para la selección de colores de la barra de menú y tipografía; y un componente para la ruta de la foto de perfil.

- VentanaConfiguracion._seleccionar_color_menu()

Objetivo: Permitir la selección personalizada del color de la barra de menú mediante herramientas nativas del sistema gráfico.

Funcionamiento Específico: Invoca el diálogo nativo de selección de colores (colorchooser.askcolor), captura el valor hexadecimal seleccionado y actualiza dinámicamente el color de fondo del botón representativo en la interfaz.

- VentanaConfiguracion._seleccionar_color_letra()

Objetivo: Permitir la personalización del color tipográfico de la aplicación.

Funcionamiento Específico: Despliega el selector nativo de colores, captura el código de color elegido por el usuario y modifica visualmente el estilo del botón correspondiente en el formulario.

- VentanaConfiguracion._seleccionar_foto()

Objetivo: Facilitar la carga de una imagen de perfil desde el sistema de archivos local mediante selectores gráficos de archivos.

Funcionamiento Específico: Abre un diálogo de exploración de archivos (filedialog.askopenfilename) filtrado por extensiones de imagen permitidas (.png, .jpg, .jpeg, .bmp) y actualiza la etiqueta visual con la ruta del archivo seleccionado.

- VentanaConfiguracion._guardar_cambios()

Objetivo: Validar los datos ingresados en el formulario y disparar el proceso de persistencia.

Funcionamiento Específico: Extrae los valores de los diferentes componentes visuales, valida que el tamaño de fuente corresponda estrictamente a un número entero y empaqueta la información en un diccionario estructurado. Acto seguido, invoca al método de guardado del gestor de persistencia, notificando al usuario el éxito o el fallo de la operación mediante mensajes emergentes.

- AplicacionPrincipal.__init__(gestor_persistencia)

Objetivo: Inicializar la ventana principal de la aplicación y coordinar la carga inicial de los ajustes.

Funcionamiento Específico: Configura las dimensiones de la ventana principal, ejecuta la carga inicial de configuración a través del gestor (garantizando el uso de valores por defecto si el archivo no existe) y manda a construir la barra de menú, la interfaz de estado y los estilos visuales iniciales.

- AplicacionPrincipal._crear_menu()

Objetivo: Configurar la barra de menús principal con opciones funcionales y simuladas.

Funcionamiento Específico: Crea un objeto Menu que contiene las opciones solicitadas: un menú Archivo con subopciones simuladas y opción de salida; un menú Edición simulado; un menú Ver simulado; y un menú Settings funcional que enlaza directamente con la apertura de la ventana de configuración.

- AplicacionPrincipal._crear_interfaz_principal()

Objetivo: Mostrar de forma clara el estado actual de la configuración cargada en el sistema.

Funcionamiento Específico: Genera un contenedor visual con un área de texto de solo lectura destinada a reflejar los parámetros activos de usuario, nombre, tema, idioma, fuentes, colores y ruta de imagen.

- AplicacionPrincipal._actualizar_vista_estado(config)

Objetivo: Refrescar el contenido textual informativo en la ventana principal tras una modificación de ajustes.

Funcionamiento Específico: Limpia el componente de texto y formatea los nuevos valores recibidos del diccionario de configuración para mantener sincronizada la interfaz con el estado actual del sistema.

- AplicacionPrincipal._aplicar_configuracion_visual(config)

Objetivo: Aplicar de manera dinámica las preferencias visuales configuradas por el usuario.

Funcionamiento Específico: Extrae los parámetros de color y diseño del diccionario de configuración y los aplica sobre los componentes gráficos de la ventana principal.

- AplicacionPrincipal._abrir_settings()

Objetivo: Controlar la apertura del módulo de configuración de usuario.

Funcionamiento Específico: Instancia la clase VentanaConfiguracion, pasando como argumentos la referencia de la ventana principal, el gestor de persistencia y la función de retorno (callback) para actualizar la vista al guardar cambios.

- AplicacionPrincipal._on_configuracion_actualizada(nueva_config)

Objetivo: Sincronizar el estado interno de la ventana principal tras un almacenamiento exitoso de preferencias.

Funcionamiento Específico: Actualiza la variable de configuración local de la clase y dispara los métodos de actualización de la vista de estado y de los componentes visuales.

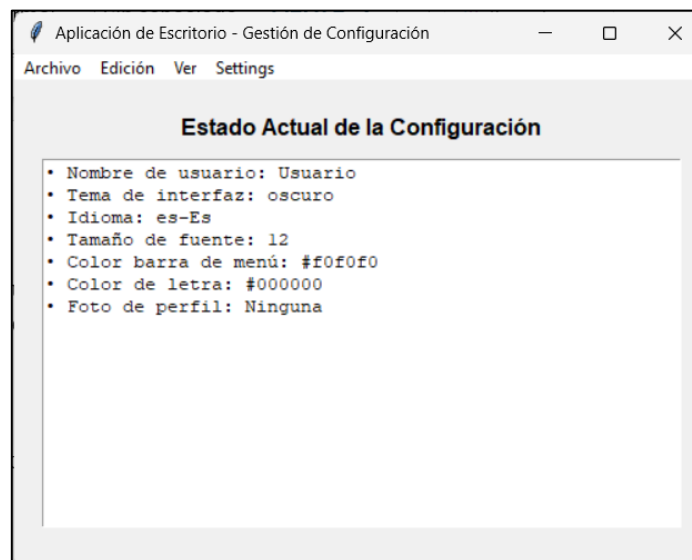
- AplicacionPrincipal._accion_simulada(nombre_accion)

Objetivo: Manejar la interacción sobre los elementos del menú que poseen un carácter ilustrativo o simulado.

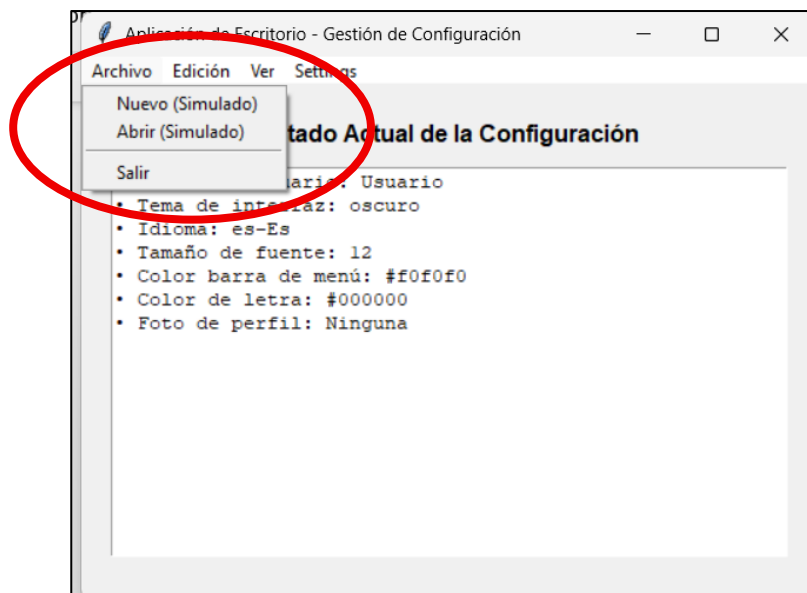
Funcionamiento Específico: Despliega una ventana emergente informativa indicando que la opción seleccionada forma parte del menú simulado requerido por las especificaciones del sistema.

Capturas de Pantalla del Sistema Funcional.

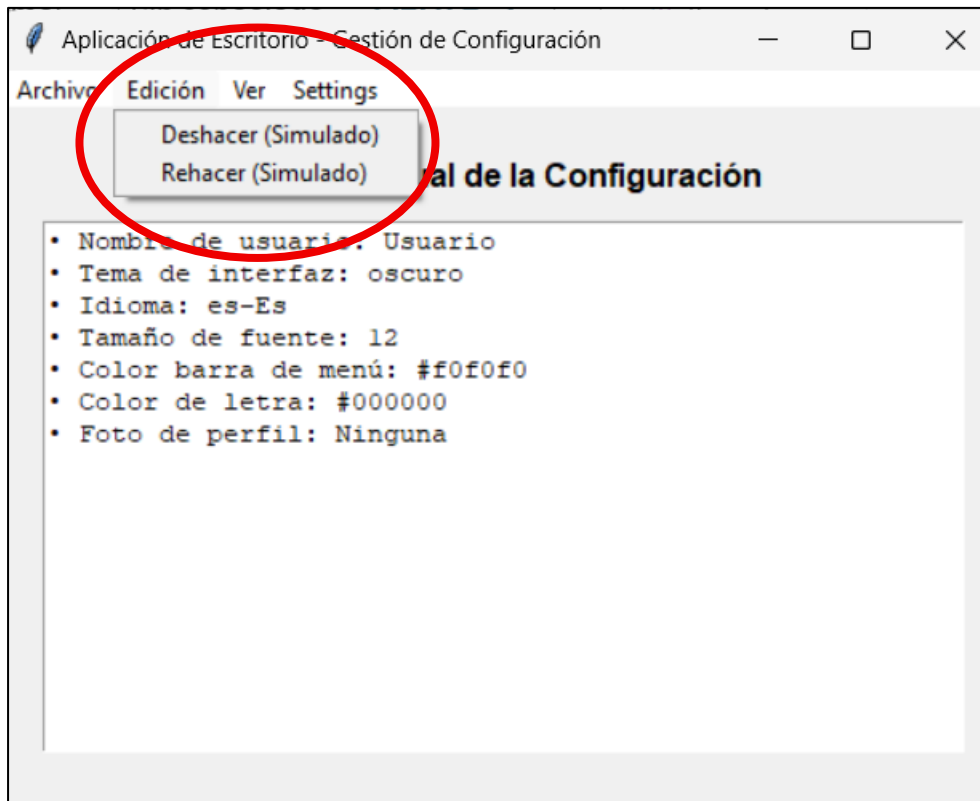
Ventana Principal:



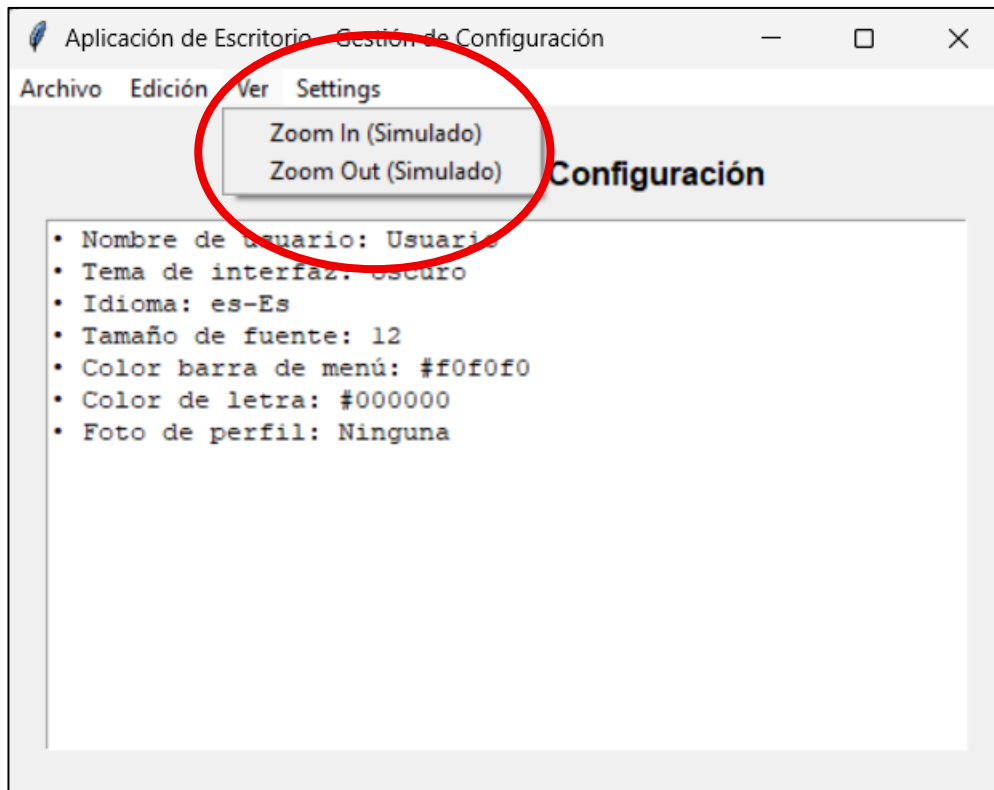
Pestaña Archivo:



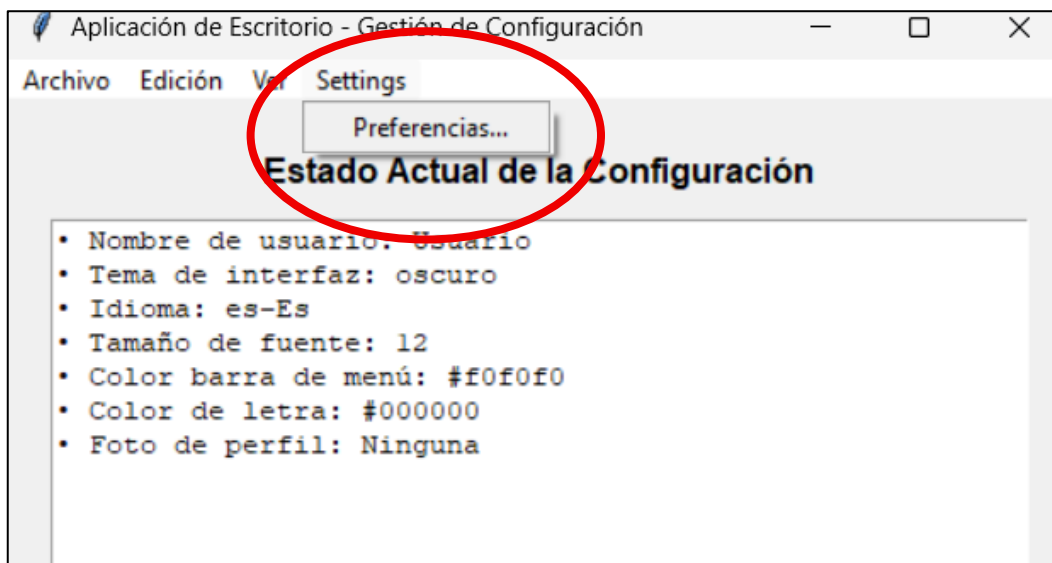
Pestaña Edición:



Pestaña Ver:



Pestaña Settings (Ajustes):



Vista dentro de Settings (Ajustes) se hizo click en la opción de preferencias y dentro de preferencias se encuentra el panel para realizar los cambios de la persistencia de datos que se guardarán en el archivo JSON.

